



FUNCTIONS II



External Variables

- Local variables can only be accessed in the function in which they are defined.
- If a variable is defined outside any function at the same level as function definitions, it is available to all the functions defined below in the same source file
 - *external variable*
- Global variables: external variables defined before any function definition
 - *Their scope will be the whole program*

Local Variables

Output:

5

5

6

5

5

6

```
/* an example of local variables */  
#include <stdio.h>
```

```
void func (void);
```

```
int main (void)  
{  
    int i = 5;  
    printf("%d \n", i);  
    func();  
    printf("%d \n", i);  
    func();  
    return 0;  
}
```

```
void func (void)  
{  
    int i = 5;  
    printf("%d \n", i);  
    i++;  
    printf("%d \n", i);  
}
```

Global Variables

Output:

5

5

6

6

6

7

```
/* an example of global variables */  
#include <stdio.h>
```

```
int i = 5;
```

```
void func (void);
```

```
int main (void)  
{  
    printf("%d \n", i);  
    func();  
    printf("%d \n", i);  
    func();  
    return 0;  
}
```

```
void func (void)  
{  
    printf("%d \n", i);  
    i++;  
    printf("%d \n", i);  
}
```

Static Variables

- A variable is said to be static if it is allocated storage at the beginning of the program execution and the storage remains allocated until the program execution terminates.
- External variables are always static
- Within a block, a variable can be specified to be static by using the keyword `static` before its type declaration:
 - *static type variable-name;*
- Variable declared static can be initialized only with constant expressions (if not, its default value is zero)

Static Variables

Output:

5

5

6

5

6

7

```
/* an example of static variables */  
#include <stdio.h>
```

```
void func (void);
```

```
int main (void)  
{  
    int i = 5;  
    printf("%d \n", i);  
    func();  
    printf("%d \n", i);  
    func();  
    return 0;  
}
```

```
void func (void)  
{  
    static int i = 5;  
    printf("%d \n", i);  
    i++;  
    printf("%d \n", i);  
}
```

Math Library Functions

- Math library functions
 - *perform common mathematical calculations*
 - *#include <math.h>*

Function	Description	Example
<code>sqrt(x)</code>	square root of x	<code>sqrt(900.0)</code> is 30.0 <code>sqrt(9.0)</code> is 3.0
<code>cbrt(x)</code>	cube root of x (C99 and C11 only)	<code>cbrt(27.0)</code> is 3.0 <code>cbrt(-8.0)</code> is -2.0
<code>exp(x)</code>	exponential function e^x	<code>exp(1.0)</code> is 2.718282 <code>exp(2.0)</code> is 7.389056
<code>log(x)</code>	natural logarithm of x (base e)	<code>log(2.718282)</code> is 1.0 <code>log(7.389056)</code> is 2.0
<code>log10(x)</code>	logarithm of x (base 10)	<code>log10(1.0)</code> is 0.0 <code>log10(10.0)</code> is 1.0 <code>log10(100.0)</code> is 2.0
<code>fabs(x)</code>	absolute value of x as a floating-point number	<code>fabs(13.5)</code> is 13.5 <code>fabs(0.0)</code> is 0.0 <code>fabs(-13.5)</code> is 13.5
<code>ceil(x)</code>	rounds x to the smallest integer not less than x	<code>ceil(9.2)</code> is 10.0 <code>ceil(-9.8)</code> is -9.0

Function	Description	Example
<code>floor(x)</code>	rounds x to the largest integer not greater than x	<code>floor(9.2)</code> is 9.0 <code>floor(-9.8)</code> is -10.0
<code>pow(x, y)</code>	x raised to power y (x^y)	<code>pow(2, 7)</code> is 128.0 <code>pow(9, .5)</code> is 3.0
<code>fmod(x, y)</code>	remainder of x/y as a floating-point number	<code>fmod(13.657, 2.333)</code> is 1.992
<code>sin(x)</code>	trigonometric sine of x (x in radians)	<code>sin(0.0)</code> is 0.0
<code>cos(x)</code>	trigonometric cosine of x (x in radians)	<code>cos(0.0)</code> is 1.0
<code>tan(x)</code>	trigonometric tangent of x (x in radians)	<code>tan(0.0)</code> is 0.0



```
#include <stdio.h>
#include <math.h>
int main()
{
    float i=5.1, j=5.9, k=-5.4, l=-6.9;
    printf("floor of %.2f is %f\n", i, floor(i));
    printf("floor of %.2f is %f\n", j, floor(j));
    printf("floor of %.2f is %f\n", k, floor(k));
    printf("floor of %.2f is %f\n", l, floor(l));
    return 0;
}
```

floor of 5.10 is 5.000000

floor of 5.90 is 5.000000

floor of -5.40 is -6.000000

floor of -6.90 is -7.000000



```
#include <stdio.h>
#include <math.h>
int main()
{
    float i=5.1, j=5.9, k=-5.4, l=-6.9;
    printf("ceil of %.2f is %f\n", i, ceil(i));
    printf("ceil of %.2f is %f\n", j, ceil(j));
    printf("ceil of %.2f is %f\n", k, ceil(k));
    printf("ceil of %.2f is %f\n", l, ceil(l));
    return 0;
}
```

ceil of 5.10 is 6.000000

ceil of 5.90 is 6.000000

ceil of -5.40 is -5.000000

ceil of -6.90 is -6.000000

Random Number Generation

- The element of chance can be introduced into computer applications by using the C standard library function `rand` from the `<stdlib.h>` header.
- Consider the following statement:
 - *`i = rand();`*
- The `rand` function generates an integer between 0 and `RAND_MAX` (a symbolic constant defined in the `<stdlib.h>` header).

Random Number Generation

- Standard C states that the value of RAND_MAX must be at least 32767, which is the maximum value for a two-byte (i.e., 16-bit) integer.
- If rand truly produces integers at random, every number between 0 and RAND_MAX has an equal chance (or probability) of being chosen each time rand is called.
- The range of values produced directly by rand is often different from what's needed in a specific application.

Random Number Generation

- For example, a dice-rolling program that simulates a six-sided dice would require random integers from 1 to 6
- We use the remainder operator (%) in conjunction with rand as follows
rand() % 6

to produce integers in the range 0 to 5.

- This is called scaling.

Random Number Generation

- The number 6 is called the scaling factor.
- We then shift the range of numbers produced by adding 1 to our previous result

$1 + \text{rand()} \% 6$

Random Number Generation



```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    for (int i = 1; i <= 20; i++)
    {
        printf("%5d", 1 + rand() % 6);
        if (i % 5 == 0)
            printf("\n");
    }
    return 0;
}
```

Output:

6	6	5	5	6
5	1	1	5	3
6	6	2	4	2
6	2	3	4	1

Random Number Generation

- Executing that program again produces exactly the same sequence of values.
- How can these be random numbers? Ironically, this repeatability is an important characteristic of function `rand`.
- Function `rand` actually generates pseudorandom numbers.
- Calling `rand` repeatedly produces a sequence of numbers that appears to be random.
- However, the sequence repeats itself each time the program is executed.

Random Number Generation

- Once a program has been thoroughly debugged, it can be conditioned to produce a different sequence of random numbers for each execution.
- This is called randomizing and is accomplished with the standard library function **srand**.
- Function **srand** takes an unsigned integer argument and seeds function **rand** to produce a different sequence of random numbers for each execution of the program.

Random Number Generation



```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    int seed;
    printf("Enter a seed:");
    scanf("%d", &seed);
    srand(seed);
    for (int i = 1; i <= 10; i++)
    {
        printf("%5d", 1 + rand() % 6);
        if (i % 5 == 0)
            printf("\n");
    }
    return 0;
}
```

Random Number Generation

Enter seed: **67**

6	1	4	6	2
1	6	1	6	4

Enter seed: **867**

2	4	6	1	6
1	1	3	6	2

Enter seed: **67**

6	1	4	6	2
1	6	1	6	4

Random Number Generation

- To randomize without entering a seed each time, use a statement like *srand(time(NULL));*
- This causes the computer to read its clock to obtain the value for the seed automatically.
- Function **time** returns the number of seconds that have passed since midnight on January 1, 1970.
- This value is converted to an unsigned integer and used as the seed to the random number generator.
- The function prototype for time is in **<time.h>**.

Random Number Generation



```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
int main(void)
{
    srand(time(NULL));
    for (int i = 1; i <= 10; i++)
    {
        printf("%5d", 1 + rand() % 6);
        if (i % 5 == 0)
            printf("\n");
    }
    return 0;
}
```

Recursion

- The programs we've discussed are generally structured as functions that call one another in a disciplined, hierarchical manner.
- For some types of problems, it's useful to have functions call themselves.
- A recursive function is a function that calls itself either directly or indirectly through another function.
- Recursion is a complex topic discussed at length in upper-level computer science courses.
- In this section and the next, simple examples of recursion are presented.

Recursion

- Recursive problem-solving approaches have a number of elements in common.
- A recursive function is called to solve a problem.
- The function actually knows how to solve only the simplest case(s), or so-called base case(s).
- If the function is called with a base case, the function simply returns a result.
- If the function is called with a more complex problem, the function divides the problem into two conceptual pieces: a piece that the function knows how to do and a piece that it does not know how to do.

Recursion

- To make recursion feasible, the latter piece must resemble the original problem, but be a slightly simpler or smaller version.
- Because this new problem looks like the original problem, the function launches (calls) a fresh copy of itself to go to work on the smaller problem—this is referred to as a recursive call or the recursion step.
- The recursion step also includes the keyword return, because its result will be combined with the portion of the problem the function knew how to solve to form a result that will be passed back to the original caller.
- The recursion step executes while the original call to the function is paused, waiting for the result from the recursion step.

Recursion

- The recursion step can result in many more such recursive calls, as the function keeps dividing each problem it's called with into two conceptual pieces.
- For the recursion to terminate, each time the function calls itself with a slightly simpler version of the original problem, this sequence of smaller problems must eventually converge on the base case.
- When the function recognizes the base case, returns a result to the previous copy of the function, and a sequence of returns ensues all the way up the line until the original call of the function eventually returns the final result to main.

Recursion

Recursively Calculating Factorials

- The factorial of a nonnegative integer n , written $n!$ (pronounced “ n factorial”), is the product

$$n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot 1$$

with $1!$ equal to 1, and $0!$ defined to be 1.

- A recursive definition of the factorial function is arrived at by observing the following relationship:

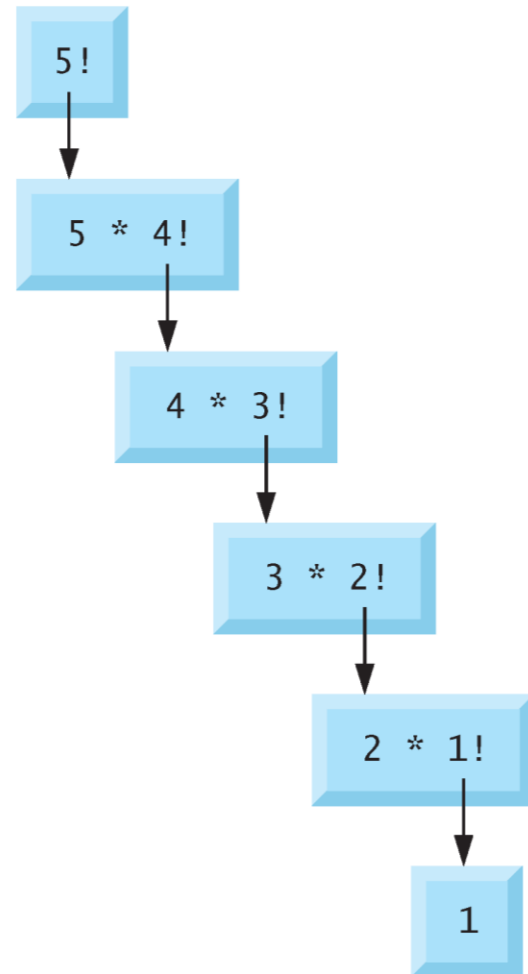
$$n! = n \cdot (n - 1)!$$

- For example, $5!$ is clearly equal to $5 * 4!$ as is shown by the following:

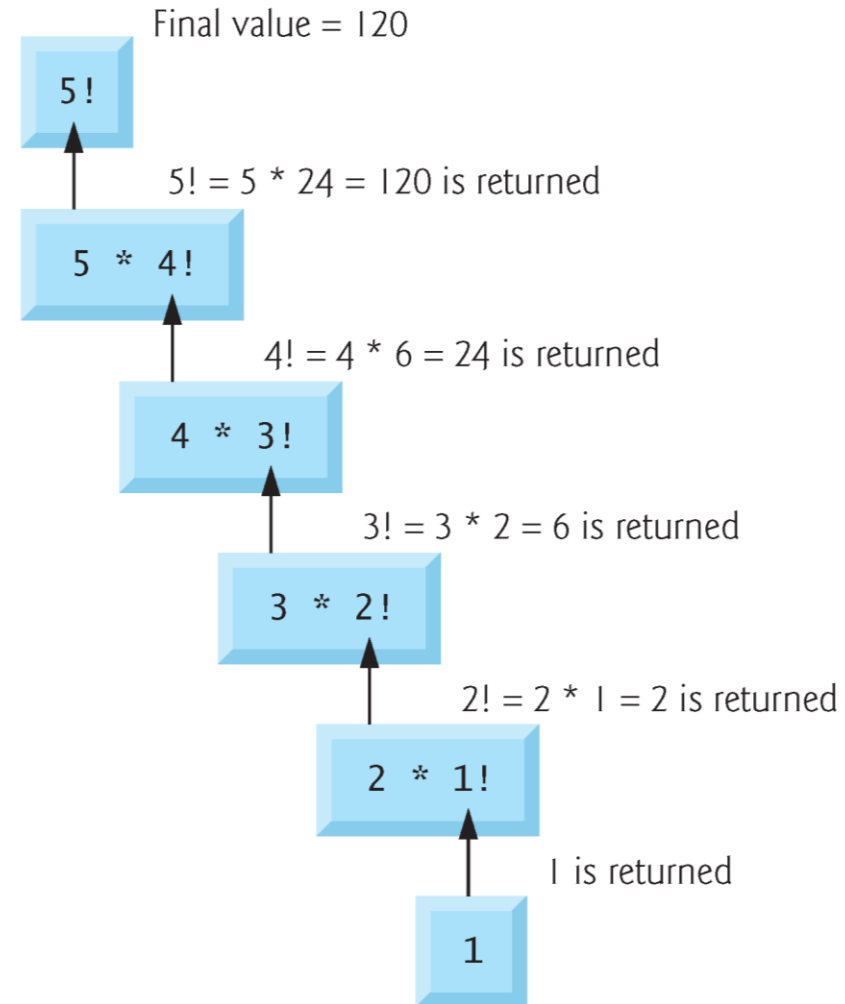
$$\begin{aligned} 5! &= 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 \\ 5! &= 5 \cdot (4 \cdot 3 \cdot 2 \cdot 1) \\ 5! &= 5 \cdot (4!) \end{aligned}$$

Recursion

a) Sequence of recursive calls



b) Values returned from each recursive call



Output:

1!=1

2!=2

3!=6

4!=24

5!=120

6!=720

7!=5040

8!=40320

9!=362880

10!=3628800

```
/* Recursive factorial function */
#include <stdio.h>
int factorial(int n);
int main(void)
{
    for (int i = 1; i <= 10; i++)
        printf("%d!= %d\n", i, factorial(i));
    return 0;
}
int factorial(int n)
{
    if (n <= 1)        //base case
        return 1;
    else                //recursive step
        return(n * factorial(n - 1));
}
```